Практическое занятие 4 "Управление состоянием приложения"

Практическое занятие №4. Управление состоянием приложения

Ветка репозитория: `pz24`

---

1. ЦЕЛЬ РАБОТЫ

Освоение методов глобального управления состоянием в React: Context API и хук `useReducer`. Решение проблемы prop drilling, реализация глобального хранилища, оптимизация перерисовок.

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

- Проп-дриллинг и способы его решения.
- Context API (`createContext`, `Provider`, `useContext`).
- Хук `useReducer` (reducer, actions, dispatch).
- Комбинация Context + useReducer как замена Redux.

---

3. ЗАДАЧИ РАБОТЫ (индивидуально)

- Создать React-приложение с несколькими компонентами, где данные передаются через props (показать проблему prop drilling).
- Реализовать глобальное состояние через Context API для хранения темы (светлая/тёмная) и языка.
- Реализовать **основную бизнес-логику** вашего курсового проекта (корзину товаров, избранное, список задач, прогресс и т.п.) с помощью `useReducer` + Context.
- Обеспечить доступ к состоянию и функциям `dispatch` в любом компоненте.

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ (общий для всех вариантов)

Часть 1. Демонстрация проблемы prop drilling

- Создайте иерархию компонентов (например, `App → Header → Navigation → UserMenu → UserAvatar`), передавая данные через все уровни.
- Покажите, что данные (например, `userName`) передаются через компоненты, которые их не используют.

Часть 2. Создание контекста темы

```
const ThemeContext = createContext();
const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState('light');
  const toggleTheme = () => setTheme(prev => prev === 'light' ? 'dark' : 'light');
  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};
```

Часть 3. Реализация бизнес-логики через useReducer + Context

Создайте редьюсер, соответствующий вашей предметной области (корзина, избранное, список задач и т.п.):

```
const cartReducer = (state, action) => {
  switch (action.type) {
    case 'ADD_ITEM':
      // ...
    case 'REMOVE_ITEM':
      // ...
    default:
      return state;
  }
};
const CartProvider = ({ children }) => {
  const [cart, dispatch] = useReducer(cartReducer, []);
  return <CartContext.Provider value={{ cart, dispatch }}>{children}</CartContext.Provider>;
};
```

Часть 4. Использование в компонентах

```
const { theme, toggleTheme } = useContext(ThemeContext);
const { cart, dispatch } = useContext(CartContext);
```

Часть 5. Сохранение состояния (опционально, рекомендуется)

- Сохраняйте корзину/избранное в `localStorage` при каждом изменении.
- Восстанавливайте состояние при загрузке приложения.

---

5. СОДЕРЖАНИЕ ОТЧЕТА

- Титульный лист, цель работы.
- Демонстрация проблемы prop drilling (схема или код).
- Листинг провайдеров (`ThemeProvider`, `CartProvider`) и редьюсера.
- Скриншоты работы: переключение темы, выполнение бизнес-операций (добавление/удаление).
- Ответы на контрольные вопросы.
- Выводы.

---

6. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.
- Ссылка на репозиторий GitHub с кодом (ветка **`pz24`**).
- **Индивидуальное выполнение. Вариант Темы** определяется по **Генератору вариантов.**

---

7. КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое prop drilling? Какие проблемы он создаёт?
2. Как создать и использовать Context API? Опишите шаги.
3. В чём преимущество `useReducer` перед `useState` для сложной логики?
4. Как комбинировать Context и useReducer для глобального состояния? Напишите пример.
5. Как предотвратить лишние перерисовки при изменении контекста?
6. Для каких данных (глобальное состояние) рекомендуется использовать Context API?
7. Как разделить Context на несколько (например, `ThemeContext`, `CartContext`, `UserContext`)?
8. Что такое `useMemo` и как он может оптимизировать значение, передаваемое в Context?
9. Как в редьюсере обработать действие, которое зависит от текущего состояния (например, добавить элемент, если его нет)?

10. Как реализовать сохранение состояния корзины в `localStorage` и восстановление при загрузке?
11. В чём разница между `useState` и `useReducer`? Когда что выбрать?
12. Как передать `dispatch` от `useReducer` через Context, чтобы дочерние компоненты могли отправлять действия?
13. Что такое «действия» (actions) в `useReducer`? Как они должны быть структурированы?
14. Как в редьюсере обработать несколько полей состояния (например, объект `{ cart, user, filters }`)?
15. Как избежать мутации состояния в редьюсере? Приведите правильные и неправильные примеры.
16. Как в React компоненте подписаться только на часть глобального состояния (селекторы)?
17. Как использовать несколько `useReducer` в одном компоненте (для разных сущностей)?
18. Как реализовать функцию «сброса состояния» (reset) в редьюсере?
19. Как отладить изменения состояния, отправляемые через dispatch?
20. Как в `useReducer` передать начальное состояние через функцию-инициализатор (lazy initialization)?
21. Как в Context провайдере мемоизировать значение с помощью `useMemo`?
22. Что такое `action creator`? Как его использовать с `useReducer`?
23. Как организовать глобальное состояние для аутентификации пользователя (логин/логаут) с помощью `useReducer`?
24. Как реализовать «отмену действия» (undo) в `useReducer` (сохранить историю состояний)?
25. Можно ли использовать `useReducer` без Context (только для локального состояния)? Приведите пример.
26. Сравните Context + useReducer и Redux (преимущества/недостатки для небольших проектов).
27. Как в компоненте получить доступ только к `dispatch`, не подписываясь на всё состояние?
28. Что такое «thunk» и как имитировать асинхронные действия в `useReducer`?
29. Как исправить проблему избыточных перерисовок при изменении контекста (например, разделить Context на два: для данных и для dispatch)?

30. Как протестировать редьюсер (чистую функцию) без React-компонентов?

---

8. КРИТЕРИИ ОЦЕНКИ

- Реализация двух независимых контекстов (тема/язык и бизнес-логика).
- Демонстрация решения prop drilling.
- Корректная работа редьюсера (добавление, удаление, изменение количества).
- Сохранение состояния в `localStorage` (рекомендуется).
- Выполнение бизнес-логики, соответствующей теме курсового проекта.
- Полнота отчёта: скриншоты, листинги, ответы на 30 вопросов.
- Ссылка на репозиторий с веткой `pz24` и инструкцией по запуску.